Davante

20

ENTORNOS DE DESARROLLO

Herramientas *Cloud* para control de versiones

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Repositorios en la nube 4

/ 3. GitHub 4

/ 4. Caso práctico 1: “Copias de seguridad” 5

/ 5. GitLab 6

/ 6. Otros repositorios Cloud 6

/ 7. Caso práctico 2: “GitLab vs GitHub” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer los principios de los repositorios en la nube.

Conocer el servicio GitLab.

Saber utilizar el servicio GitHub.

Conocer otros servicios Cloud.

/ 1. Introducción y contextualización práctica

El desarrollo *software* cada vez cuenta con mayores servicios remotos basados en almacenamiento y computación en la nube. Como no podría ser menos, los servicios de alojamiento de código han ido creciendo en los últimos años para dar soporte y solución a posibles desarrolladores independientes que no cuentan con la infraestructura local para poder almacenar el código correctamente. Además, este tipo de soluciones han permitido que múltiples empresas puedan simplificar su infraestructura local y depender de servicios externos alojando el código en servidores remotos, aligerando así sus tareas de mantenimiento y copias de seguridad.

Cuando utilizamos un servicio basado en *Cloud*, especialmente para almacenamiento de código, es importante tener en cuenta las políticas de seguridad y privacidad que se establecen sobre el código que estamos compartiendo con otra empresa. Es importante que sepamos qué sucedería si existe un acceso no autorizado al repositorio, y también las políticas de copia de seguridad que nos ofrece nuestro proveedor del servicio. En este tema vamos a presentar las principales características de los servicios ofrecidos para establecer un repositorio en la nube. Además, nos detendremos en aquellos que son más utilizados y relevantes en la actualidad. Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.



Fig. 1. El almacenamiento en la nube puede liberar responsabilidades.

Audio Intro. "Permisos y repositorios remotos"

http://bit.ly/3q5t9KL

/ 2. Repositorios en la nube

En la actualidad los servicios en la nube han ido evolucionando significativamente incorporando cada vez más cantidad de servicios disponibles a los programadores. Gracias a esta evolución, ha sido posible poder desarrollar sistemas relativamente complejos en tiempos más reducidos. Existen diferentes tipos arquitecturas *cloud:*

- **SaaS (*Software* como Servicio).** Es posible utilizar el *software* **a través de las aplicaciones con conexiones remotas.** Por ejemplo Gmail, Calendar, etc.
- **PaaS (Plataforma como Servicio).** En este caso podemos conectar **a un sistema remoto para acceder a todos los servicios** que pueda ofrecer.
- **IaaS (Infraestructura con un Servicio).** Existen soluciones como AWS en la que tenemos una **infraestructura que podemos gestionar para poder crear los servicios que deseemos**. En este caso podríamos disponer de servidores EC2 para realizar computación y S3 para el almacenamiento.

Por otra parte, podemos destacar que los distintos servicios *clouds*, pueden presentar diferentes **tipos de acceso**: **privado, público o híbrido**. Si tenemos en cuenta algunos inconvenientes de estos tipos de nubes, podríamos resaltar que la privacidad y la protección de datos sería el mayor problema de las nubes de acceso público. Sin embargo, a cambio, este tipo de nubes permite compartir desarrollos de manera muy sencilla. Si relacionamos los repositorios con los sistemas *cloud* y sus infraestructuras y forma de acceso vemos que la infraestructura más utilizada es SaaS. Por ello, mediante este sistema, es posible compartir los servicios y dar mayor elasticidad a los recursos que se ofrezcan a los usuarios.



Fig. 2. Es necesario escoger un buen almacenamiento para nuestro código.

Audio 1. "Compartir código de forma segura"
http://bit.ly/3oEp93D

/ 3. GitHub

GitHub es una **plataforma para el desarrollo colaborativo que permite almacenar diferentes proyectos de los usuarios.** Esta plataforma utiliza un sistema de control de versiones basado en Git. Normalmente, **el código de los proyectos que desarrollamos en estas plataformas se almacena públicamente**, aunque hay una tendencia cada vez mayor a realizarlo de forma privada.

A. Características principales

Esta plataforma ofrece algunas características que requieren mención:

» Es **posible crear una *Wiki* para cada uno de los proyectos que desarrollemos.** Gracias a ella es posible poder explicar el contenido que desarrollemos y dar acceso a diferentes usuarios documentando la herramienta desarrollada o el código desarrollado.

» **Cada proyecto puede disponer de su propia página web** de manera independiente.

» Es posible **utilizar funcionalidades** como si una tarea social se tratase. Por ejemplo, ofrece la posibilidad de que los desarrolladores tengan una comunidad de seguidores.

» Ofrecen **herramientas de trabajo colaborativo** (la versión de pago amplia los servicios y utilidades).

» En el caso de estar utilizando metodologías ágiles, es posible utilizar **un gestor de proyectos como *Kanban* o parecido.**

B. Formas de trabajo

Esta plataforma ofrece un **servicio web**, a través del cual podemos **acceder a todos los servicios y gestionar el código que tengamos en el repositorio**. Normalmente, podemos acceder al portal web y realizar todas las gestiones sobre los diferentes proyectos que tengamos. Además, ofrece una herramienta de escritorio en la que podamos trabajar y gestionar también todos nuestros proyectos esta herramienta se conoce como *GitHub Desktop*.

Vídeo 1. "GitHub con GitKraken"
https://bit.ly/3s8Wupa

/ 4. Caso práctico 1: "Copias de seguridad"

Planteamiento. En nuestra empresa, el servicio de Tecnologías de la Información (IT) cada vez está más saturado, dadas las continuas tareas que se tienen que realizar en el código fuente. Por ejemplo, vemos que es necesario realizar copias de seguridad periódicas y no disponemos de la infraestructura local para poder realizarlo correctamente. Dado que el gran activo de nuestra empresa es el código fuente corremos el riesgo de poder perder todo nuestro valor.

Nudo. ¿Qué alternativas podrías plantear?

Desenlace. Según hemos visto en el tema, una posibilidad sería utilizar servicios basados en la nube para poder realizar copias de seguridad del código y no tener todo almacenado de manera local, como, por ejemplo, el servicio de IaaS de AWS.

Otra alternativa sería utilizar herramientas *Cloud* para establecer repositorio de código remoto y evitar duplicidades de éste. Si tenemos un repositorio como *GitHub* no sería necesaria realizar las copias de seguridad tan frecuentemente porque el propio sistema es capaz de establecer todas las copias que sean necesarias.

Si suscribimos un plan de pago, el nivel de copias podría ser aún mayor y tendremos mayor seguridad sobre nuestro código.

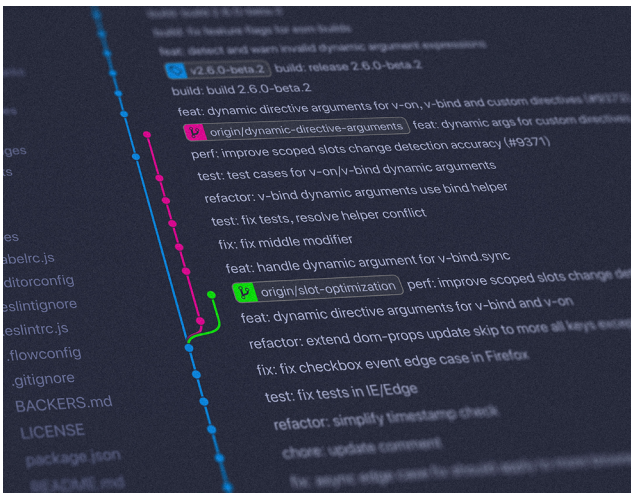


Fig. 3. Repositorio centralizado.

/ 5. GitLab

GitLab es un **servicio web que implementa un sistema de control de versiones para facilitar el trabajo colaborativo mediante la utilización de Git**. Ofrece la posibilidad de que el desarrollador cree diferentes repositorios con la capacidad de crear *wikis*, seguimiento de errores, etc.

Desde el punto de vista de la funcionalidad, es posible utilizar *GitLab* exactamente igual que *GitHub*, aunque existen algunas diferencias.

- **Diferencias entre *GitLab* y *GitHub***

  Ambos repositorios ofrecen diferentes suscripciones para poder subir el código. En la comparativa que realizamos entre *GitLab* y *GitHub* vamos a considerar únicamente **la versión gratuita** de las dos herramientas.

  » **Diferentes niveles de autenticación.** A través de ***GitLab*** podemos fijar **permisos de acceso de grano fino**, es decir, podemos fijar los permisos a nivel de archivo. Sin embargo, en el caso de ***GitHub***, los **permisos se fijan a nivel de repositorio**, pudiendo distinguir entre lectura o escritura únicamente.

  » **Integración continua.** En ***GitLab*** existe la posibilidad de realizar **integración continua** que muchas veces puede ahorrar tiempo a los equipos de desarrolladores y a nuestro equipo de calidad.

  » **Seguimiento de errores.** ***GitHub*** permite **destacar la función** para la **búsqueda y corrección de errores**.

  » **Colaboración.** ***GitHub*** permite crear **repositorios privados** de **marea ilimitada**, pero con un máximo de tres colaboradores. Sin embargo, ***GitLab*** ofrece la posibilidad de **crear repositorios privados ilimitados** en los cuales no hay límite en el número de colaboradores.

  » **Tamaño máximo.** Existe una diferencia en el tamaño que puede alcanzar el repositorio. Mientras que ***GitHub*** ofrece hasta **1GB** por repositorio, con un tamaño máximo de archivos de 100 MB, ***GitLab*** ofrece un tamaño de **10 GB** por repositorio.



Vídeo 2. "Mi primer proyecto en GitLab"
https://bit.ly/3osP8Ld



/ 6. Otros repositorios Cloud

Como hemos comentado anteriormente, existen diferentes **alternativas a la hora de elegir repositorio *Cloud* de código**. En los últimos años, han ido apareciendo nuevas implementaciones y soluciones que permiten ampliar el número de opciones disponibles y mejorar las diferentes ofertas existentes. Hay que tener en cuenta que todas estas plataformas *Cloud* ofrecen diferentes tipos de planes para poder almacenar el código en ellas. Desde el punto de vista de nuestra empresa, el servicio o la modalidad elegida, **va a depender también si queremos compartir el código con empresas externas.** Además, debemos **considerar los aspectos de seguridad** que puedan tener implementados para poder crear nuestro código. Algunas alternativas son:

- ***Bitbucket*.** En los últimos años se ha convertido en una alternativa bastante potente a la hora de competir con otras plataformas. Ofrece la posibilidad de **crear repositorios privados de manera gratuita y permite importar proyectos desde GitHub**. Es compatible con herramientas muy utilizadas como Jira. Además, en el plan gratuito es posible añadir hasta cinco colaboradores a los repositorios.

- ***SourceForge***. Se trata de un **repositorio de código** que utiliza publicidad para poder mantenerse. En los últimos años ha rediseñado el sitio web para ser más atractiva y fácil de usar.

- ***GNU Savannah***. Es una plataforma totalmente de código libre que permite realizar el control de versiones con múltiples herramientas como Git, SVN, etc. Su objetivo es servir de soporte para los proyectos que se basen en únicamente el código libre.

- ***Gitea***. Se trata de un **repositorio que utiliza Git**. Posee características similares a *GitHub, BitBucket y GitLab*.

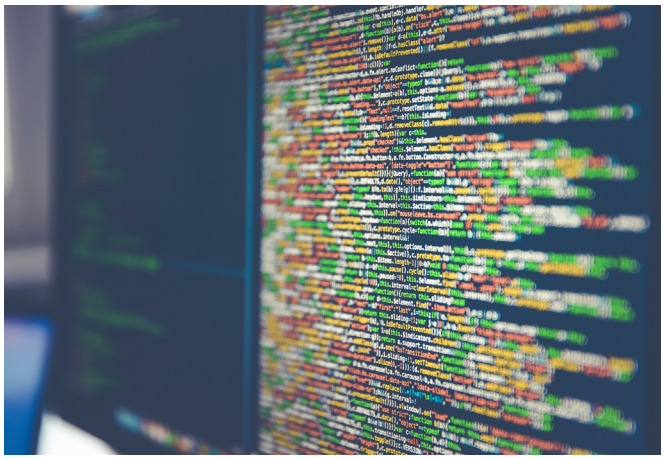


Fig. 4. Debemos elegir la mejor opción para salvaguardar nuestro código.

/ 7. Caso práctico 2: "GitLab vs GitHub"

Planteamiento: En nuestra empresa se plantea la necesidad de crear un proyecto en el que vamos a compartir el código con una empresa externa y con la que aún no hemos firmado el documento de confidencialidad. Sospechamos que el nivel de confidencialidad no va a ser suficientemente grande y no nos interesa tener todo el proyecto compartido con esta empresa. Por lo tanto, tenemos que ser capaces de determinar qué parte del proyecto compartir. Durante más de 10 años hemos estado trabajando con servicios basados en Cloud y recientemente nos hemos planteado migrar el repositorio de código a otro sistema porque el nuestro no permite compartir código.

Nudo: ¿Qué alternativas plantearías?

Desenlace. Como hemos visto en el tema, las plataformas más utilizadas como *GitHub y GitLab* permiten compartir el código con diferentes colaboradores, incluso en los planes gratuitos.

Sin embargo, el nivel de granularidad que nos ofrecen es diferente. Como se nos plantea en el caso práctico, es importante que podamos elegir qué parte del repositorio del código compartir con nuestro nuevo cliente. Para poder hacer esto lo recomendable sería utilizar un repositorio del código que permita fijar una granularidad fina a la hora de establecer los permisos. Por ejemplo, podríamos utilizar *GitLab* porque nos permite determinar qué parte de un repositorio compartir con nuestros colaboradores. Sin embargo, no sería adecuado utilizar *GibHub* porque el nivel de permisos que podemos establecer es a nivel de repositorio y no de archivos.



Fig. 5. GitLab permite establecer permisos con más detalle.

/ 8. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema hemos presentado las principales características que ofrecen los **servicios basados en *Cloud*** para el almacenamiento de código utilizando repositorios remotos. Para ello hemos presentado diferentes tipos de arquitecturas *Cloud* y a partir de ellas, destacado algunas de las herramientas y empresas que se dedican a la creación de repositorio remotos.

Además, hemos analizado dos de los más importantes en la actualidad que están basados en el sistema de control de versiones Git: ***GitLab* y *GitHub***. Como hemos podido observar, ambos repositorios poseen unas características similares ofreciendo diferentes planes al usuario para que se pueda subscribir y almacenar el código en sus servidores. Existen **otros repositorios**, quizás menos conocidos, que pueden ser muy interesantes dadas las políticas de precio y las necesidades nuestra empresa.

A la hora de elegir un repositorio remoto para nuestro código es importante tener en cuenta los aspectos de seguridad que vamos a necesitar porque esto puede ser determinante a la hora de poder compartir el proyecto con clientes o colaboradores. Si realizamos una mala elección de un repositorio, podemos tener grandes problemas a la hora de migrar hacia otro. Por otra parte, muchas empresas ven en este aspecto una oportunidad de negocio y ofrecen facilidades para poder migrar de un repositorio a otro.

Resolución del caso práctico de la unidad

Como hemos visto en este tema, los repositorios remotos pueden tener diferentes tipos de permisos a la hora de otorgárselos a los usuarios que se conectan al mismo. Es posible que tengamos permiso de lectura, pero no de escritura sobre el repositorio en el que estamos trabajando. Una solución sería pedir permiso a la empresa con la que estamos trabajando para que nos dé permiso de escritura. Sin embargo, esto puede ser una tarea muy peligrosa ya que si realizamos algún cambio que tenga consecuencias negativas, afectaría al funcionamiento normal del código de la empresa. Una solución podría ser crearnos una propia rama de desarrollo para nosotros, y tener permiso de escritura sobre ella.

/ 9. Bibliografía

Gutiérrez, A. & Guervós, J. (2017). Aprende Git: [... y, de camino, GitHub. S.l: Independently Published.

Grady, A. (2018). GitLab quick start guide: migrate to GitLab for all your repository management solutions. Birmingham, UK: Packt Publishing.